

DPDK Mode Architecture Guide

Purpose: Technical guide to understanding how the DPDK mode IDS pipeline works, from kernel bypass packet capture to ML predictions.

Table of Contents

- 1. [High-Level Overview](#)
- 2. [What is DPDK?](#)
- 3. [Architecture Components](#)
- 4. [Key Functions Breakdown](#)
- 5. [Command-Line Usage](#)

High-Level Overview

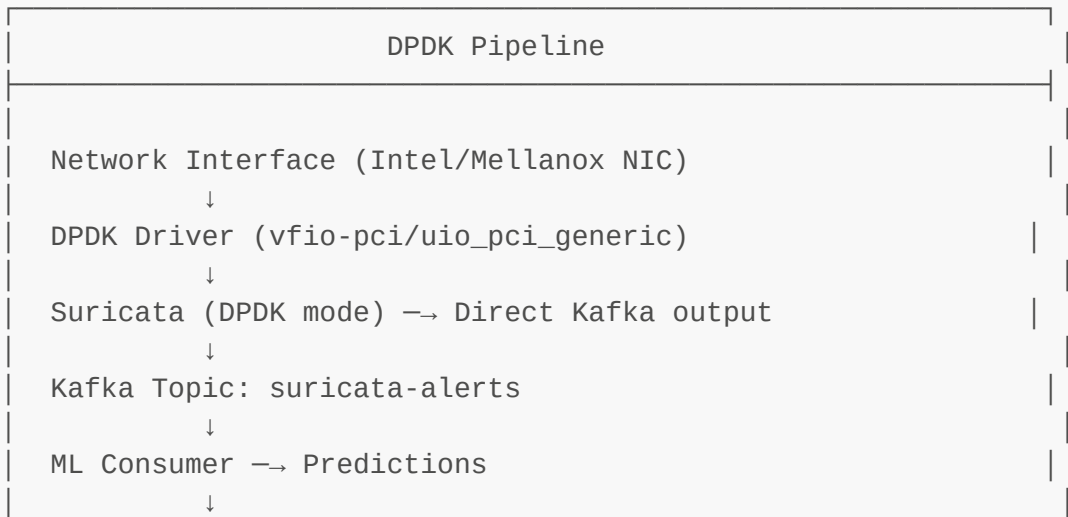
What is DPDK Mode?

DPDK (Data Plane Development Kit) is a high-performance packet processing framework that bypasses the Linux kernel for direct user-space access to network hardware. It achieves 10-100 Gbps throughput by eliminating kernel overhead and using poll-mode drivers.

Why We Use It

- ✓ **Extreme Performance:** 10-100 Gbps packet processing (vs 100-500 Mbps with AF_PACKET)
- ✓ **Low Latency:** Microsecond-level latency vs millisecond-level
- ✓ **Zero Copy:** Direct memory access (DMA) without kernel copying
- ✓ **Kernel Bypass:** Eliminates context switches and system calls
- ✓ **Production Ready:** Used by telecom and enterprise networks

Architecture at a Glance



```
Kafka Topic: ml-predictions
```

Key Characteristic: Direct streaming from Suricata to Kafka (no file intermediary needed).

What is DPDK?

Core Concepts

DPDK is a set of libraries and drivers that enable fast packet processing by:

1. **Kernel Bypass:** Packets go directly from NIC to user-space application
2. **Poll Mode Drivers (PMD):** Continuous polling instead of interrupt-driven I/O
3. **Hugepages:** Uses large memory pages (2MB vs 4KB) for better TLB performance
4. **Zero-Copy DMA:** Direct memory access without kernel buffers
5. **CPU Affinity:** Pins threads to specific CPU cores to avoid context switches

How DPDK Works

Traditional Kernel Path (AF_PACKET):

NIC → Kernel Driver → Interrupt → System Call → User Space (100-500 Mbps)

└ Context Switch ─┘ └ Memory Copy ─┘

DPDK Kernel Bypass Path:

NIC → DPDK Driver → Poll → User Space (10-100 Gbps)

└ Direct Memory Access (DMA) ─┘

Why DPDK is Faster:

- ✗ **No interrupts:** Continuous polling eliminates interrupt overhead
- ✗ **No context switches:** Stays in user space
- ✗ **No memory copies:** Zero-copy DMA transfers
- ✓ **Batch processing:** Processes multiple packets per cycle
- ✓ **Cache optimization:** CPU cache-friendly data structures

Supported Network Interface Cards (NICs)

Fully Supported (Production-Ready)

Intel NICs

- 82599ES - 10 Gigabit Ethernet
- X710/XL710 - 10/40 Gigabit Ethernet
- E810 - 25/100 Gigabit Ethernet

- Performance: 10-100 Gbps
- Drivers: ixgbe, i40e, ice

Mellanox NICs

- ConnectX-4 - 25/100 Gigabit Ethernet
- ConnectX-5 - 100 Gigabit Ethernet
- ConnectX-6 - 200 Gigabit Ethernet
- Performance: 25-200 Gbps
- Drivers: mlx4, mlx5

Broadcom NICs

- BCM57xxx series - 10/25 Gigabit Ethernet
- Performance: 10-25 Gbps
- Driver: bnxt

Partially Supported

Vendor	Models	Notes
Virtual NICs	virtio, vmxnet3	For VMs and testing
Amazon ENA	Elastic Network Adapter	AWS EC2 instances
Microsoft	Hyper-V vmbus	Azure VMs

× NOT Compatible:

- USB Ethernet adapters
- Consumer-grade Realtek NICs (RTL8111/8168)
- WiFi adapters
- Basic integrated laptop NICs

—
PROF

DPDK Driver Types

DPDK requires binding NICs to special user-space drivers:

1. vfio-pci (Recommended)

- **Type:** Kernel module with IOMMU support
- **Security:** Hardware-level memory isolation
- **Use Case:** Production deployments
- **Requirements:** IOMMU/VT-d enabled in BIOS
- **Command:** `dpdk-devbind.py --bind=vfio-pci 0000:02:00.0`

2. uio_pci_generic (Testing)

- **Type:** Generic kernel UIO driver
- **Security:** Basic isolation

- **Use Case:** Development and testing
- **Requirements:** Included in Linux kernel
- **Command:** `dpdk-devbind.py --bind=uio_pci_generic 0000:02:00.0`

3. igb_uio (Legacy)

- **Type:** Custom DPDK kernel module
- **Security:** Moderate isolation
- **Use Case:** Older systems without VFIO
- **Requirements:** Must compile separately
- **Status:** Deprecated, avoid if possible

Hardware Requirements

Minimum:

- Intel/Mellanox/Broadcom NIC (see supported list)
- 4 CPU cores (2 for DPDK, 2 for system)
- 4GB RAM + 2GB hugepages
- IOMMU/VT-d support (for vfio-pci)

Recommended:

- 8+ CPU cores
- 16GB RAM + 4GB hugepages
- Intel X710 or Mellanox ConnectX-5 NIC
- NUMA-aware configuration

Architecture Components

1. Network Interface Layer

Component: Intel/Mellanox NIC

Example: Intel X710 (10/40GbE), Mellanox ConnectX-5 (100GbE)

Configuration Requirements:

- **Driver Binding:** Must be bound to DPDK driver (vfio-pci/uio_pci_generic)
- **PCI Address:** Identified by PCI slot (e.g., `0000:02:00.0`)
- **State:** Interface goes OFFLINE for normal networking (DPDK exclusive)
- **Hugepages:** 1024 × 2MB pages (2GB total) allocated

Binding Process:

```
# 1. Unbind from kernel driver
dpdk-devbind.py --unbind 0000:02:00.0

# 2. Bind to DPDK driver
```

```
dpdk-devbind.py --bind=vfio-pci 0000:02:00.0
```

```
# 3. Verify binding  
dpdk-devbind.py --status
```

2. DPDK Driver Layer

Component: DPDK Poll Mode Driver (PMD)

Managed By: Suricata with DPDK support

How DPDK Drivers Work:

```
// Conceptual C code showing DPDK initialization  
  
// 1. Initialize EAL (Environment Abstraction Layer)  
rte_eal_init(argc, argv);  
  
// 2. Allocate memory pools for packet buffers  
struct rte_mempool *mbuf_pool;  
mbuf_pool = rte_pktmbuf_pool_create("MBUF_POOL",  
    NUM_MBUEFS, CACHE_SIZE, 0, RTE_MBUF_DEFAULT_BUF_SIZE,  
    rte_socket_id());  
  
// 3. Configure NIC port  
struct rte_eth_conf port_conf = {  
    .rxmode = {  
        .mq_mode = ETH_MQ_RX_RSS, // Receive Side Scaling  
    },  
};  
rte_eth_dev_configure(port_id, rx_rings, tx_rings, &port_conf);  
  
// 4. Setup RX/TX queues  
rte_eth_rx_queue_setup(port_id, queue_id, nb_rx_desc, socket_id,  
    &rx_conf, mbuf_pool);  
  
// 5. Start device  
rte_eth_dev_start(port_id);  
  
// 6. Poll for packets (continuous loop)  
while (1) {  
    nb_rx = rte_eth_rx_burst(port_id, queue_id, pkts, BURST_SIZE);  
    for (i = 0; i < nb_rx; i++) {  
        process_packet(pkts[i]);  
    }  
}
```

DPDK Features Used:

- **RSS (Receive Side Scaling):** Distributes packets across multiple RX queues based on hash
- **Multi-Queue:** Each CPU core gets dedicated RX/TX queue
- **Burst Processing:** Processes batches of packets (e.g., 32 at a time)
- **Mempool:** Pre-allocated packet buffer pool for zero-copy

Configuration in Suricata:

```
# /etc/suricata/suricata.yaml

dppk:
  eal-params:
    proc-type: primary
    log-level: info

  interfaces:
    - interface: 0000:02:00.0 # PCI address
      threads: 4 # Worker threads
      promisc: yes # Promiscuous mode
      multicast: yes
      mtu: 1500
      mempool-size: 262144 # Packet buffer pool
      rx-queues: 4 # RX queues (matches threads)
      tx-queues: 4 # TX queues
```

Performance Tuning:

- **Threads:** Set to number of CPU cores dedicated to DPDK
- **Mempool Size:** Larger = handles bursts better (256K typical)
- **RX Queues:** One per worker thread for parallel processing
- **RSS:** Distributes flows across queues for load balancing

3. Hugepages (Memory Management)

PROF

Component: Linux Hugepage Filesystem

Location: `/mnt/huge` or `/dev/hugepages`

Why Hugepages?

- Standard pages: 4KB → TLB can cache ~2048 pages (8MB address space)
- Hugepages: 2MB → TLB can cache ~2048 pages (4GB address space)
- Result: 500x better TLB hit rate = faster memory access

Configuration:

```
# Allocate 1024 × 2MB hugepages (2GB total)
echo 1024 > /proc/sys/vm/nr_hugepages
```

```
# Mount hugepage filesystem
mkdir -p /mnt/huge
mount -t hugetlbfs nodev /mnt/huge

# Verify allocation
cat /proc/meminfo | grep Huge
# HugePages_Total:    1024
# HugePages_Free:     512
# Hugepagesize:       2048 kB
```

Memory Layout:

Standard Memory (4KB pages):



Hugepages (2MB pages):



4. Suricata IDS Engine (DPDK Mode)

Component: Suricata IDS with DPDK support

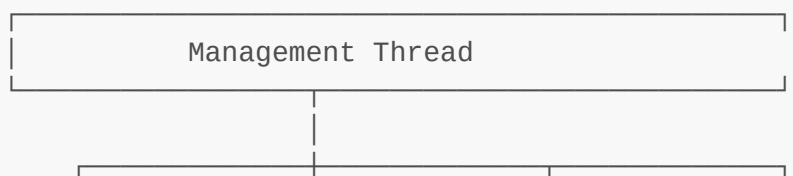
Version: 7.0.x (compiled with `--enable-dpdk`)

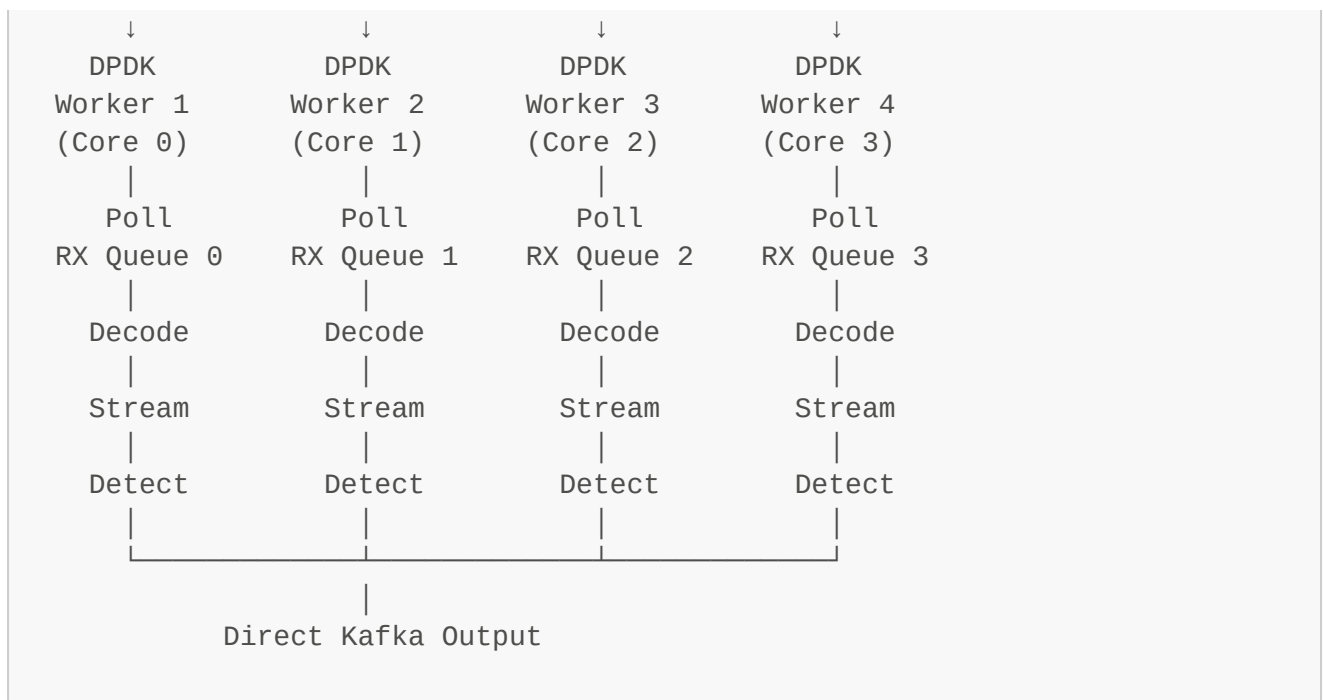
Role: High-speed deep packet inspection with kernel bypass

What Suricata Does (DPDK Mode):

1. **DPDK Initialization:** Binds to NIC, allocates hugepages, creates mempools
2. **Packet Reception:** Polls NIC continuously using `rte_eth_rx_burst()`
3. **Batch Processing:** Processes 32-64 packets per poll cycle
4. **Protocol Parsing:** Decodes packets (Ethernet → IP → TCP/UDP → Application)
5. **Flow Tracking:** Maintains state for connections (same as AF_PACKET)
6. **Signature Matching:** Applies threat detection rules
7. **Kafka Output:** Streams events directly to Kafka (no file!)

Multi-Threading Architecture (DPDK):





Key Differences from AF_PACKET Mode:

- **✓ No interrupts:** Continuous polling (100% CPU usage)
- **✓ Batch processing:** Processes multiple packets per cycle
- **✓ Zero-copy:** Packets stay in DPDK mempool
- **✓ Direct Kafka output:** No eve.json file or bridge needed
- **✓ NUMA awareness:** Memory allocated on same NUMA node as NIC

Suricata Configuration File (`/etc/suricata/suricata.yaml`):

```
# DPDK-specific sections

dppk:
  eal-params:
    proc-type: primary
    log-level: info

  interfaces:
    - interface: 0000:02:00.0
      threads: 4
      promisc: yes
      mempool-size: 262144
      rx-queues: 4

# Threading (CPU affinity)
threading:
  set-cpu-affinity: yes
  cpu-affinity:
    - management-cpu-set:
        cpu: ["0"]
    - receive-cpu-set:
        cpu: ["1", "2", "3", "4"]
```

```
- worker-cpu-set:
  cpu: ["1", "2", "3", "4"]
```

```
# Direct Kafka output (no file)
```

```
outputs:
```

```
- kafka:
  enabled: yes
  brokers: ["localhost:9092"]
  topic: suricata-alerts
```

Performance Characteristics:

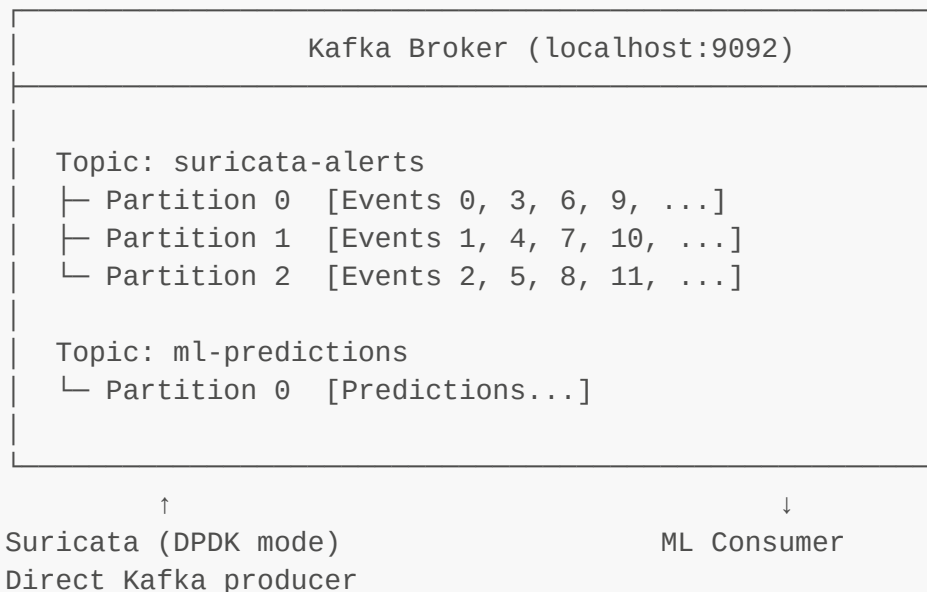
- **Throughput:** 10-100 Gbps (depending on NIC)
- **Latency:** < 100 microseconds per packet
- **CPU Usage:** 100% on worker cores (polling)
- **Memory:** 2-4GB hugepages + standard RAM

5. Apache Kafka (Message Broker)

Component: Apache Kafka 3.6.0

Role: Same as AF_PACKET mode - distributed streaming platform

Architecture (Same as AF_PACKET):



PROF

Key Difference: No Kafka bridge needed - Suricata writes directly to Kafka!

Topic Configuration (Same as AF_PACKET):

```
kafka-topics.sh --create \
  --topic suricata-alerts \
```

```
--partitions 3 \  
--replication-factor 1 \  
--config retention.ms=86400000 \  
--config compression.type=gzip
```

6. ML Consumer (Machine Learning Inference)

Component: Python script `ml_kafka_consumer.py`

Role: Same as AF_PACKET mode - real-time threat classification

What It Does (Same as AF_PACKET):

1. Consumes events from `suricata-alerts` Kafka topic
2. Extracts 65 CICIDS2017 features
3. Runs ML models (Random Forest, LightGBM, etc.)
4. Adaptive ensemble prediction
5. Publishes to `ml-predictions` topic

No Changes Needed: ML consumer is identical for both modes - it only cares about Kafka events, not how they were generated.

Key Functions Breakdown

Master Control Script: `run_dpdk_mode.sh`

1. Initialization & Checks

```
print_header()
```

- Displays banner showing DPDK mode
- Indicates high-performance mode
- Warns about NIC compatibility requirements

```
check_root()
```

- Verifies script is run with `sudo`
- **Why needed:** DPDK requires root for:
 - Hugepage allocation
 - Driver binding/unbinding
 - Memory mapping

```
load_config()
```

- Loads settings from `dpdk_suricata_ml_pipeline/config/pipeline.conf`
- Reads variables like:
 - `DPDK_INTERFACE` - PCI address (e.g., `0000:02:00.0`)
 - `DPDK_DRIVER` - Driver to use (vfiopci/uio_pci_generic)
 - `HUGEPAGES` - Number of hugepages to allocate
 - `DPDK_CORES` - CPU cores for DPDK workers

```
check_dependencies()
```

- Validates required software:
 - **Suricata** - Must be compiled with `--enable-dpdk`
 - **dpdk-devbind.py** - Driver binding tool
 - **Kafka** - Message broker
 - **Python3** - For ML consumer
- Checks hugepage support in kernel
- Verifies IOMMU/VT-d enabled (for vfio-pci)

```
check_hugepages()
```

- **DPDK-Specific Function**
- Verifies hugepages are allocated
- If not, prompts user and allocates 1024 × 2MB pages
- Mounts hugepage filesystem at `/mnt/huge`
- **Implementation:**

```
HUGEPAGES=$(cat /proc/meminfo | grep HugePages_Free | awk '{print $2}')
if [ "$HUGEPAGES" -lt 1024 ]; then
    echo "Allocating hugepages..."
    echo 1024 > /proc/sys/vm/nr_hugepages
    mkdir -p /mnt/huge
    mount -t hugetlbfs nodev /mnt/huge
fi
```

```
check_dpdk_binding()
```

- **DPDK-Specific Function**
- Checks if interface is bound to DPDK driver
- Uses `dpdk-devbind.py --status` to verify

- Returns PCI address and driver info
- **Example check:**

```
dpdk-devbind.py --status | grep "0000:02:00.0"  
# Output: 0000:02:00.0 'Intel X710' drv=vfio-pci unused=i40e
```

2. DPDK-Specific Setup Functions

`bind_interface()`

What it does:

1. Checks if already bound (skip if yes)
2. Finds PCI address of network interface
3. Unbinds from kernel driver (e.g., `i40e`, `ixgbe`)
4. Binds to DPDK driver (vfio-pci or uio_pci_generic)
5. Verifies binding succeeded
6. **Warning:** Interface goes OFFLINE for normal networking

Process:

```
# 1. Get PCI address  
PCI_ADDR=$(ethtool -i $INTERFACE | grep bus-info | cut -d' ' -f2)  
# Result: 0000:02:00.0  
  
# 2. Check current driver  
CURRENT_DRIVER=$(dpdk-devbind.py --status | grep $PCI_ADDR | awk '{print $3}')  
  
# 3. Unbind from kernel driver  
dpdk-devbind.py --unbind $PCI_ADDR  
  
# 4. Bind to DPDK driver  
dpdk-devbind.py --bind=vfio-pci $PCI_ADDR  
  
# 5. Verify  
dpdk-devbind.py --status | grep $PCI_ADDR
```

What happens:

```
Before:  
Interface: eth0  
Driver: i40e (kernel)  
State: UP (ping, SSH, web work)  
IP: 192.168.1.100
```

After:

Interface: 0000:02:00.0
Driver: vfio-pci (DPDK)
State: OFFLINE (only DPDK can use)
IP: None (no networking)

`unbind_interface()`

What it does:

1. Unbinds interface from DPDK driver
2. Binds back to kernel driver
3. Restores normal networking
4. Brings interface UP
5. Restores IP configuration

When to use:

- After testing DPDK mode
- Need interface for normal networking
- Before system shutdown
- Switching back to AF_PACKET mode

Process:

```
# 1. Unbind from DPDK driver
dpdk-devbind.py --unbind $PCI_ADDR

# 2. Bind to kernel driver
dpdk-devbind.py --bind=$KERNEL_DRIVER $PCI_ADDR

# 3. Bring interface up
ip link set $INTERFACE up

# 4. Restore IP (if needed)
ip addr add 192.168.1.100/24 dev $INTERFACE
```

`show_dpdk_info()`

What it does:

- Displays comprehensive DPDK status
- Shows all NICs and their bindings
- Lists available drivers
- Shows hugepage allocation

Output example:

```
== DPDK Information ==
```

```
Network devices using DPDK-compatible driver
```

```
=====
```

```
0000:02:00.0 'Intel X710 10G' drv=vfio-pci unused=i40e
```

```
Network devices using kernel driver
```

```
=====
```

```
0000:03:00.0 'Realtek RTL8111' if=eth1 drv=r8169 unused=
```

```
Hugepages:
```

```
HugePages_Total:      1024
```

```
HugePages_Free:       512
```

```
Hugepagesize:         2048 kB
```

3. Component Startup Functions

`start_kafka()`

Same as AF_PACKET mode - no changes needed

`start_suricata()`

Modified for DPDK:

1. Checks hugepages allocated
2. Ensures interface is bound to DPDK driver
3. Calls `03_start_suricata_dpdk.sh`
4. Starts Suricata with DPDK parameters
5. Verifies Suricata is using DPDK mode

PROF

Process check:

```
if pgrep -f "suricata.*--dpdk" > /dev/null; then
    echo "✓ Suricata running in DPDK mode"
fi
```

Key Difference: No Kafka bridge startup needed (Suricata writes directly to Kafka)

`start_ml_consumer()`

Same as AF_PACKET mode - no changes needed

4. Status & Monitoring Functions

show_status()

Enhanced for DPDK:

Displays status with DPDK-specific info:

1. DPDK Binding Status

```
if dpdk-devbind.py --status | grep -q "drv=vfio-pci"; then
    echo "✓ DPDK binding: vfio-pci"
    echo "  PCI: 0000:02:00.0"
fi
```

2. Hugepages Status

```
HUGEPAGES_FREE=$(cat /proc/meminfo | grep HugePages_Free | awk
'{print $2}')
echo "  Hugepages free: $HUGEPAGES_FREE / 1024"
```

3. Kafka Status (same as AF_PACKET)

4. Suricata Status (checks for DPDK mode specifically)

5. ML Consumer Status (same as AF_PACKET)

No Kafka Bridge Status: Not needed in DPDK mode

view_logs()

Modified for DPDK:

1. **Suricata logs** - Same as AF_PACKET
2. **ML consumer logs** - Same as AF_PACKET
3. **No Kafka bridge logs** - Not applicable in DPDK mode
4. **DPDK logs** - Shows DPDK-specific logs (EAL initialization, PMD status)

5. Shutdown Functions

stop_all()

Modified shutdown sequence for DPDK:

- | | |
|----------------|---------------------|
| 1. ML Consumer | ← Top of chain |
| 2. Suricata | ← No bridge to stop |

3. Kafka ← Infrastructure
4. (Optional) Unbind interface ← Restore networking

Key Difference: No Kafka bridge to stop

Implementation:

```
# Stop ML consumer
pkill -f "ml_kafka_consumer.py"

# Stop Suricata
pkill -f "suricata"
sleep 2 # Let DPDK cleanup properly

# Stop Kafka
bash "${PIPELINE_SCRIPTS}/stop_all.sh"

# Prompt to unbind (optional)
read -p "Unbind interface from DPDK? (y/n): " choice
if [ "$choice" = "y" ]; then
    unbind_interface
fi
```

Why optional unbinding?

- If restarting soon → keep bound (faster restart)
- If done for the day → unbind (restore normal networking)
- Emergency → leave bound, manually unbind later

6. Interactive Menu System

PROF

`show_menu()`

DPDK-specific menu:

```
===== MENU =====
1) Start Complete Pipeline (Kafka + Suricata + ML)
2) Start Kafka Only
3) Start Suricata Only (DPDK mode)
4) Start ML Consumer Only
5) Check System Status
6) Bind Interface to DPDK
7) Unbind Interface from DPDK
8) View DPDK Information
9) View Logs
10) Stop All Services
```

0) Exit

Additional DPDK options:

- Option 6: Manual interface binding
- Option 7: Manual interface unbinding
- Option 8: View DPDK status details

Command-Line Usage

Interactive Mode

```
sudo ./run_dpdk_mode.sh

# Menu appears, select options:
Enter choice [0-10]: 6    # Bind interface first (one-time)
Enter choice [0-10]: 1    # Start complete pipeline
```

Non-Interactive Mode (Automation)

```
# One-time setup: Bind interface to DPDK
sudo ./run_dpdk_mode.sh bind

# Start complete pipeline
sudo ./run_dpdk_mode.sh start

# Individual components
sudo ./run_dpdk_mode.sh kafka    # Start Kafka only
sudo ./run_dpdk_mode.sh suricata # Start Suricata (DPDK mode)
sudo ./run_dpdk_mode.sh ml       # Start ML consumer only

# Status and info
sudo ./run_dpdk_mode.sh status   # Check all components
sudo ./run_dpdk_mode.sh info     # Show DPDK binding details

# View logs
sudo ./run_dpdk_mode.sh logs     # Interactive log viewer

# Shutdown
sudo ./run_dpdk_mode.sh stop     # Stop all (prompts to unbind)
sudo ./run_dpdk_mode.sh unbind   # Unbind interface only
```